

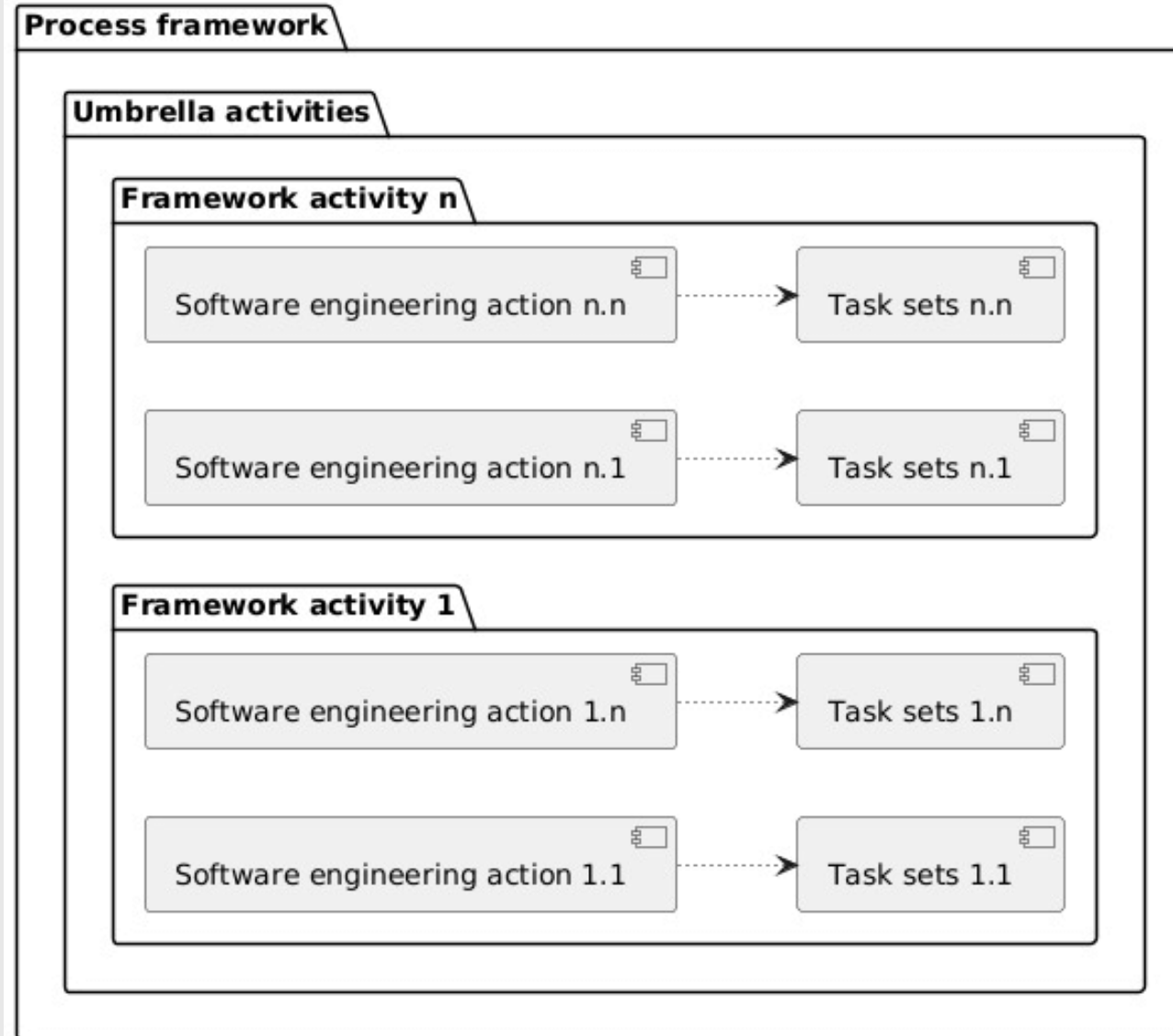
UNIT 2

Software Process Model

Generic Process Framework

- The process framework is very important in software development because it provides the base for starting the process.
- It helps define common activities that are applied to all projects, regardless of their size.
- The framework also includes umbrella activities, which are performed throughout the entire development process.
- In addition, it contains framework activities made up of specific software engineering actions.
- Each software engineering action is defined by a *task set* that identifies
 - **the work tasks that are to be completed,**
 - **the work products that will be produced,**
 - **the quality assurance points that will be required,**
 - **the milestones that will be used to indicate progress.**

Generic Process Framework



Generic Process Framework

- A generic process framework defines **five** framework activities
 - **Communication**
 - **Planning**
 - **Modeling**
 - **Construction**
 - **Deployment**
- In addition, a set of umbrella activities:
 - **project tracking and control**
 - **risk management**
 - **quality assurance**
 - **configuration management**
 - **technical reviews**
 - **and others** are applied throughout the process.

Process flow

- The arrangement (order and timing) of activities in the software process is called **process flow**.
- It explains how framework activities, along with their actions and tasks, are organized in sequence and over time.
- Process flow execution can be organized:
 1. **Linear Process Flow**
 - Activities are done one after another in order.
 2. **Iterative Process Flow**
 - Some activities are repeated before moving forward.
 3. **Evolutionary Process Flow**
 - Activities are performed in a cyclic (circular) manner.
 4. **Parallel Process Flow**
 - Multiple activities are done at the same time.
 - Example: designing one part while coding another part.

Process flow

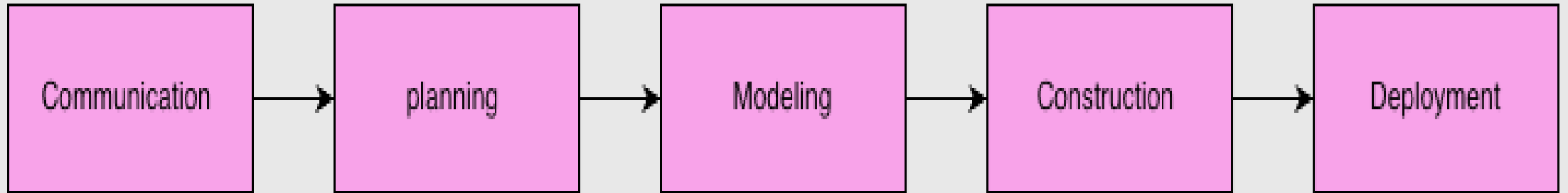


Fig: Linear flow

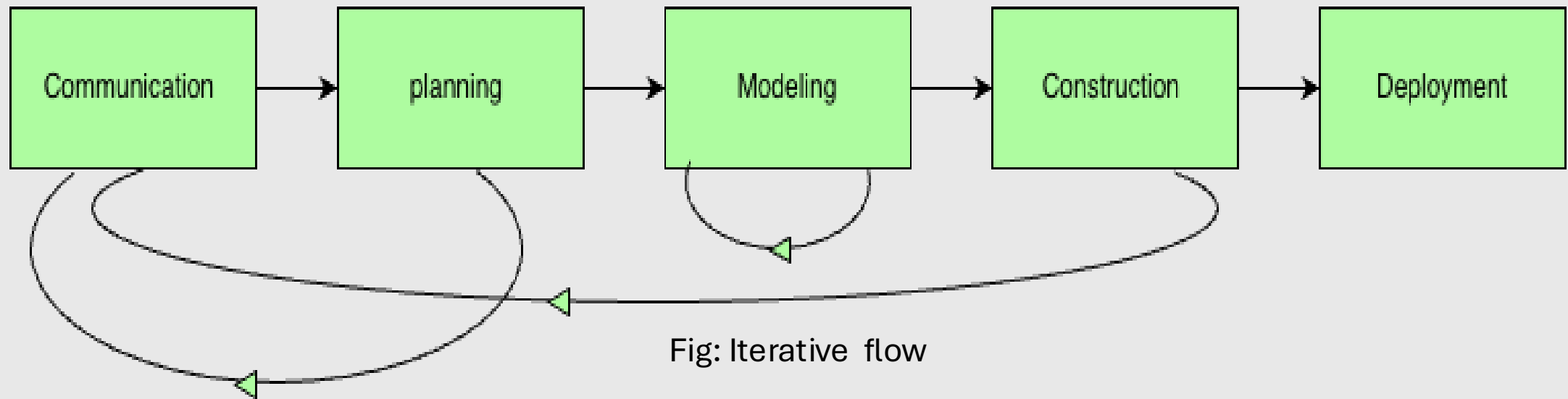


Fig: Iterative flow

Process flow

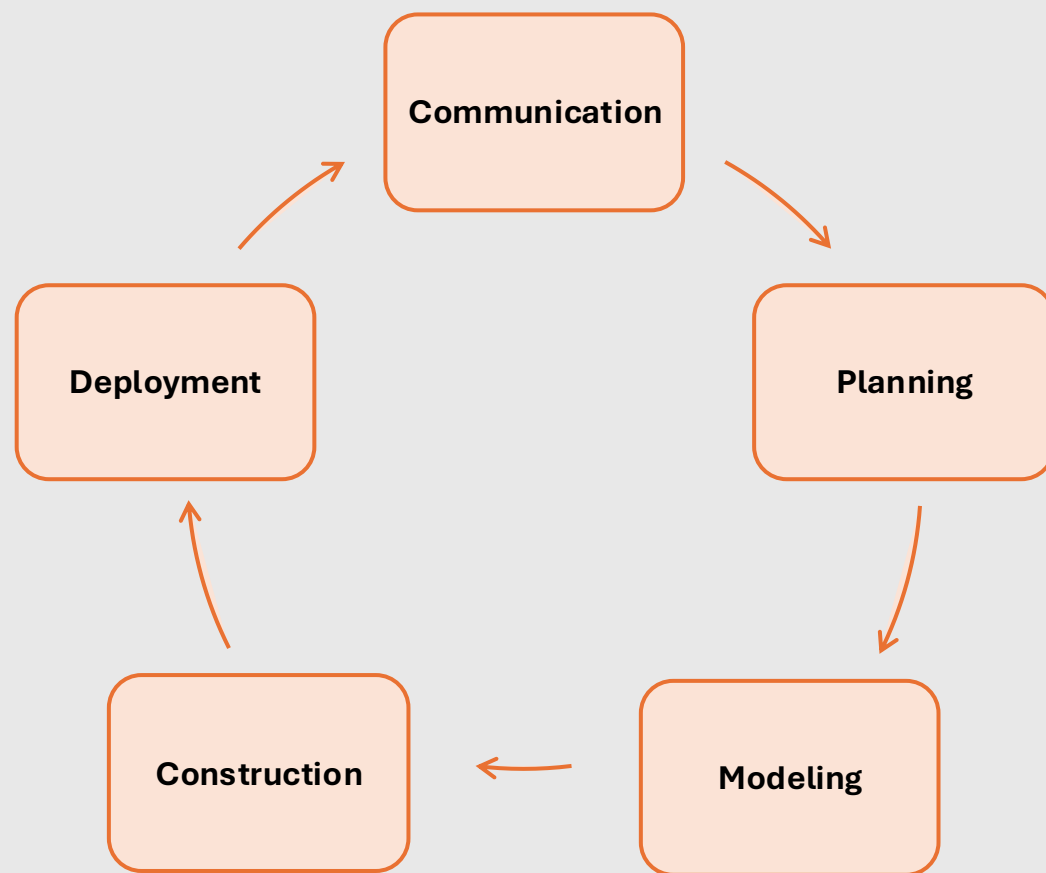


Fig: Evolutionary flow

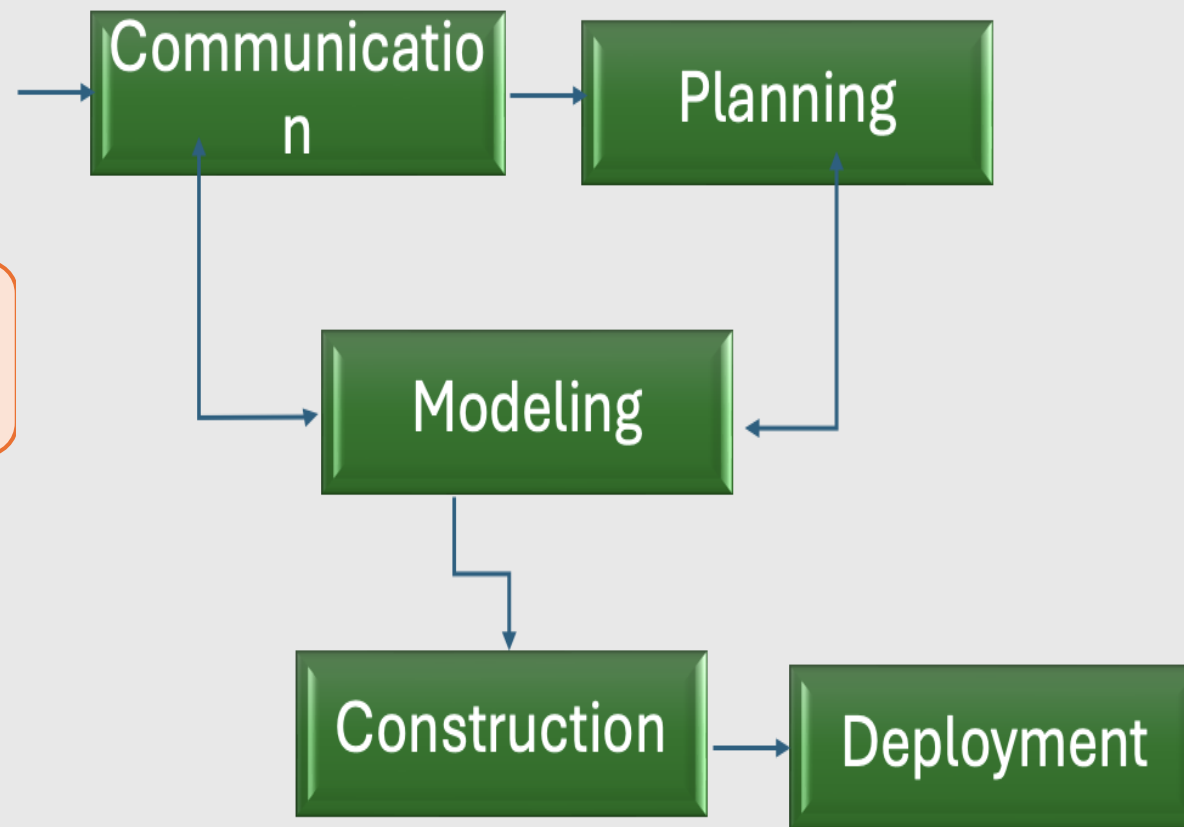


Fig: Parallel flow

Framework Activity

Defining a Framework Activity

- A framework activity needs **detailed information** before execution.
- The team must decide **what actions are required**.
- Actions depend on:
 - Nature of the problem
 - Team skills and experience
 - Stakeholders' requirements

Identifying a task set

Task set

- It is the actual list of work tasks, milestones, and deliverables.
- How do you choose one?
 - It depends on:
 - **The Problem:** Is it simple or complex?
 - **The People:** Is the team experienced or junior?
 - **The Stakeholders:** What are their specific needs or constraints?

Process pattern

- A **Process Pattern** is essentially a "proven recipe" for solving a recurring problem in software development.
- Instead of solving the same problem from scratch every time, you use a template.

The Levels of Abstraction

- Patterns can be applied at different "zoom levels" of a project:
- **Complete Process Model:** A pattern for the entire project strategy (e.g., using a *Prototyping* pattern).
- **Framework Activity:** A pattern for a specific stage (e.g., a *Planning* pattern).
- **Specific Action:** A pattern for a single task (e.g., a *Project Estimating* pattern).

System Development life cycle

- SDLC stands for “ Software Development Life Cycle” .
- SDLC is a collection of processes which are followed to develop a software.
- SDLC is a methodology that defines some processes which are followed to develop a high-quality software.
- It covers the detailed plan for building, developing and maintaining the software.
- The main aim of SDLC is to define all the tasks required for developing and maintaining software.
- It is followed for a software project within a software developing organization.

System Development life cycle

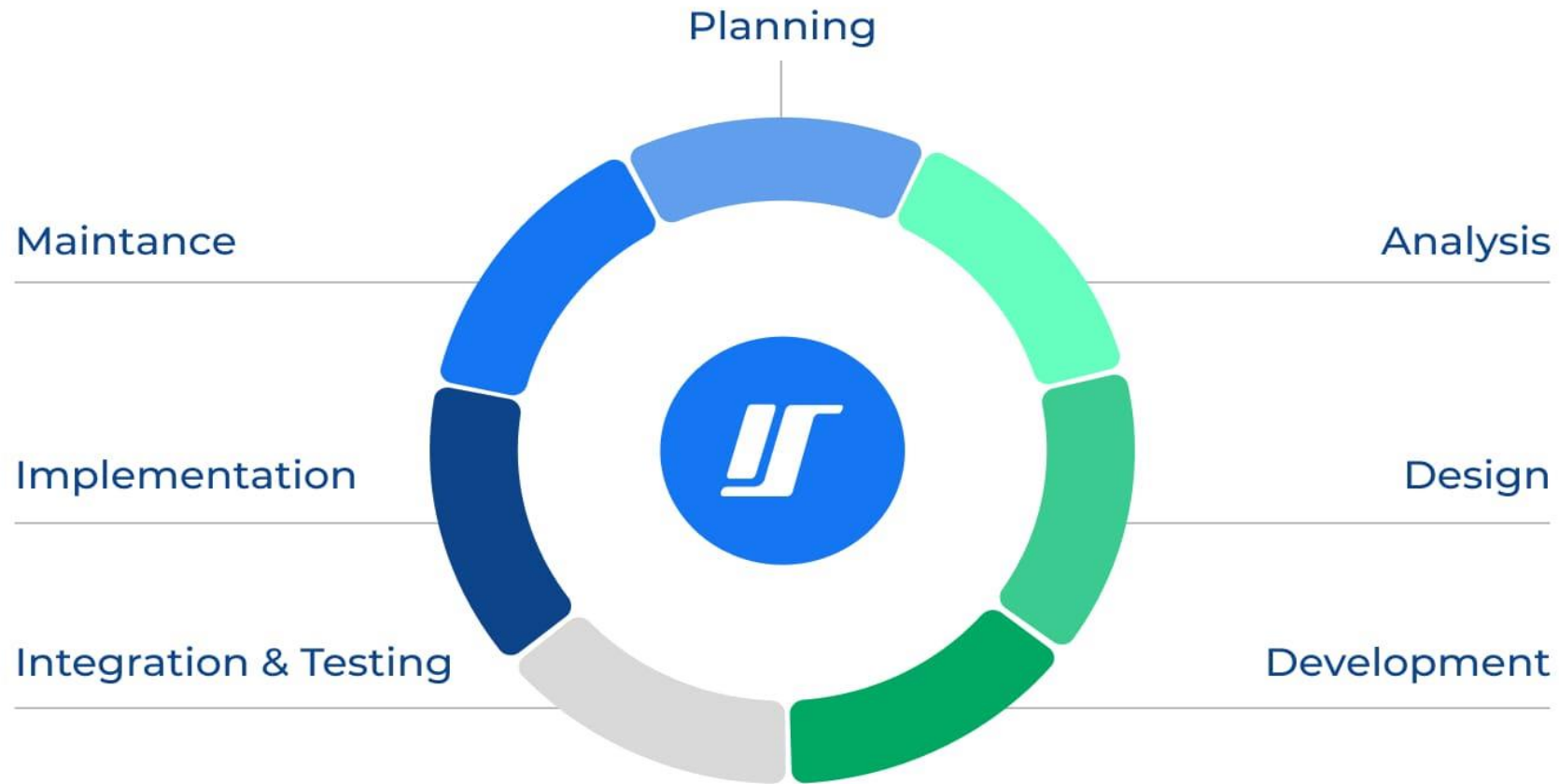
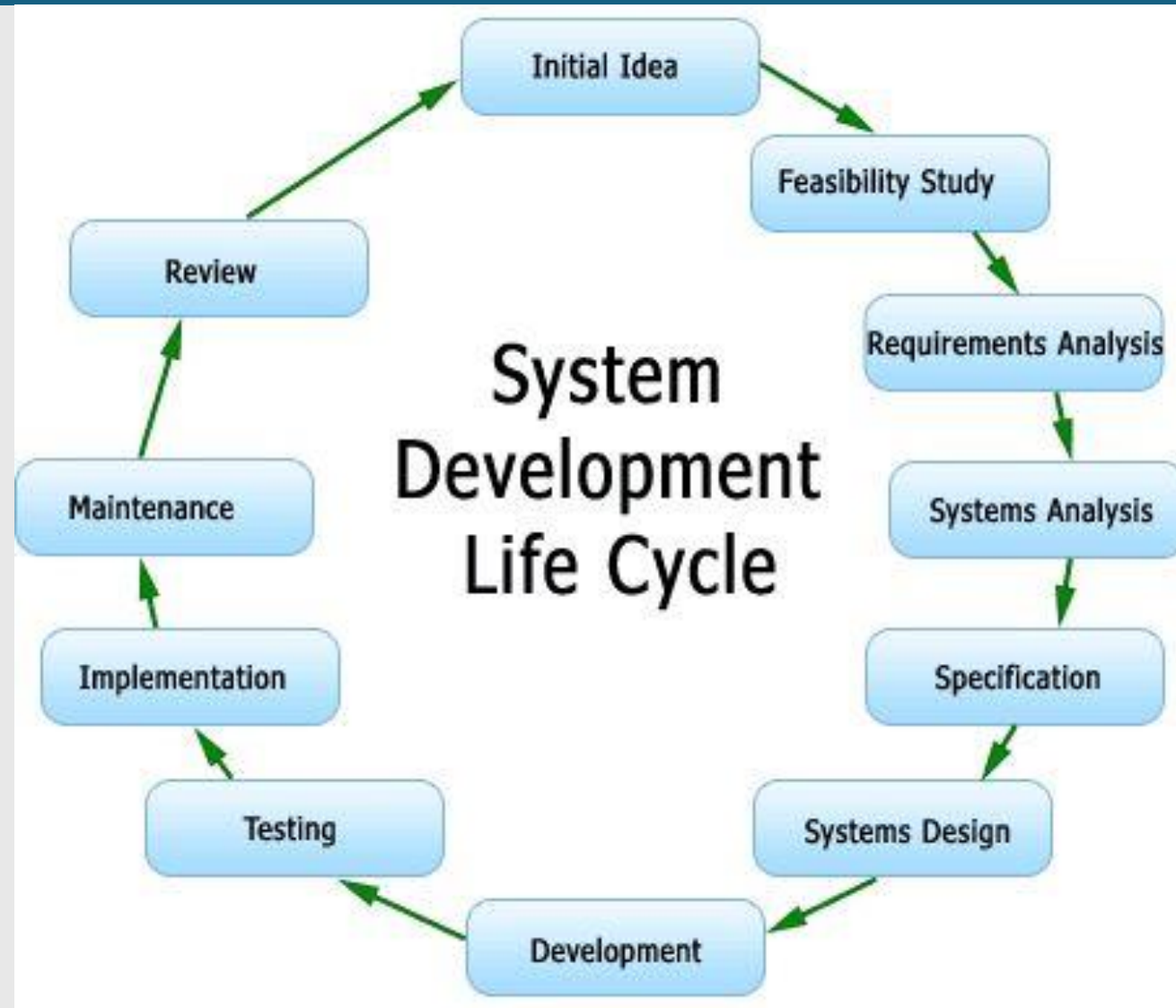


Fig : Stages of SDLC

System Development life cycle



Project initiation/ Planning

- This is the **first and most crucial phase** of the SDLC.
In this phase:
- The **need for a new system** or enhancement of an existing system is identified.
- The goal is to **understand the problem, define project scope, and set objectives.**
- During the planning Phase, a potential information system project is first identified and described -meaning the organization defines what problem it wants to solve or what improvement it wants to make.
- Then, the **system analyst evaluates** whether the project should **continue or not** by studying:
 - Is the project **technically possible?**
 - Is it **cost-effective?**
 - Will it **benefit the organization?**
- This evaluation is known as a **Feasibility Study.**

Feasibility study

- As the name suggests, a feasibility Study is carried out to determine whether a proposed project is **practical, achievable, and worth the investment**.
- It is one of the most important steps during the planning phase of the Software Development Life Cycle (SDLC).
- The main purpose of the feasibility study is to **analyze the strengths and weaknesses** of the proposed system and **decide** whether to proceed, modify, or cancel the project.
- **Objectives of Feasibility Study**
 - The feasibility study helps answer key questions such as:
 - Do we have the required **resources and technology** to develop the project?
 - Will the project be **profitable and beneficial** for the organization?
 - Can it be completed **within the given time and budget**?
 - Will it **comply with legal and operational standards**?

Feasibility study

- After the feasibility analysis, the project can be:
 - **Accepted** (if it is feasible),
 - **Accepted with modifications**, or
 - **Rejected** (if it is not feasible).

Types of Feasibility Study

1. Technical Feasibility

- This type checks whether the organization has the **technical resources and expertise** to develop the system.
- It analyzes:
 - Availability of **hardware and software**
 - **Compatibility** with existing systems
 - **Technical skills** of the development team
 - **Ease of maintenance** and future **upgrades**
- Example: Can our current servers support the new system? Do we have developers skilled in the required technology?

Feasibility study

2. Economic Feasibility (Cost-Benefit Analysis)

- This determines whether the project is **financially viable**.
- It compares the **estimated cost** of developing and maintaining the system with the **expected benefits** (profits or savings).
- It helps decide whether the investment in the project is justified.
- Example: Will the new system increase productivity enough to recover its cost?

3. Legal Feasibility

- This examines whether the proposed system **complies with all legal and regulatory requirements**.
- It ensures that no law, regulation, or copyright is violated during development or operation.
- Example: Does the system comply with data protection and privacy laws?

Feasibility study

4. Operational Feasibility

- This checks whether the system will **function effectively** once implemented.
- It focuses on how well the system meets **user requirements** and whether the **organization's staff** can operate it.
- Example: Will users be comfortable using the new interface and workflows?

5. Scheduling Feasibility (Time Feasibility)

- This assesses whether the project can be **completed within the given time frame**.
- It evaluates the organization's **time constraints, resources, and project schedule**.
- Example: Can we complete the system before the financial year ends?

Importance / Benefits of Feasibility Study

- A feasibility study is conducted to determine the **success probability** of a proposed project before investing resources such as **time, money, and manpower**.
- It helps organizations make **informed decisions** and **reduce project risks**.

Key Benefits

- **Evaluates project success probability**
 - Helps to get a clear idea of whether the project is likely to succeed before investing in it.
- **Ensures efficient resource allocation**
 - Avoids wasting budget, manpower, and time on projects that are not feasible.
- **Improves project team focus**
 - Provides a clear direction and scope for the project, allowing teams to stay focused on achievable goals.

Importance / Benefits of Feasibility Study

- **Supports “Go / No-Go” decision**
 - Provides valuable insights and data to help management decide whether to proceed with or cancel the project.
- **Identifies valid reasons for the project**
 - Confirms that the project addresses a genuine business need or problem.
- **Helps in diagnosing errors early**
 - Detects potential problems or risks before the project begins, aiding in early troubleshooting.
- **Increases success rate**
 - By evaluating multiple parameters (technical, economic, operational, legal, and scheduling), it enhances the overall chances of project success.

Requirement and Analysis

- The **Requirement and Analysis Phase** is one of the most crucial stages of the **Software Development Life Cycle (SDLC)**. It involves gathering detailed information from the customer and analyzing it to understand what the software should do and how it should function.
- In this phase, all necessary information is collected from the customer to develop the software according to their needs and expectations. The development team and stakeholders discuss key questions such as:
 - What is the need for the software?
 - Who will be the end user?
 - What is the future scope of the software?

Requirement and Analysis

- In this phase, the goal is to **deeply understand the organization's current system and define exactly what the new system should do.**
- The objective of this phase is to **collect and analyze detailed information** about customer requirements.
- **Main Objectives of the Analysis Phase**
 - To understand the **existing system** (if one exists).
 - To identify the **problems** or limitations in the current system.
 - To **collect and analyze requirements** from users and stakeholders.
 - To **document those requirements** clearly in the form of an **SRS (System Requirement Specification)**.
 - To **propose possible solutions or system alternatives** that can meet the user's needs.

Design

- The Design Phase focuses on converting the requirements defined in the SRS (System Requirement Specification) into a detailed system design that serves as a blueprint for development.
- **Process**
 - The recommended alternative solution from the analysis phase is transformed into **logical** and **physical** system specifications.
 - All aspects of the system are designed — including **input and output screens, reports, databases, and processing logic**.
- **Logical Design**
 - Describes **what** the system must do. i.e., like what data it needs, what processes it will perform, and how information will flow.
 - It is **independent of hardware or software platforms**.
 - It focuses on defining system functions and data flow without considering implementation details.

Design

- **Physical Design**

- Describes **how** the system will be implemented.
- Converts the logical design into **technology-specific details** (hardware, software, database design, and user interface).
- It provides the actual blueprint for coding and system construction.

- **Role of SRS**

- The **SRS** serves as a key reference document for system designers to create the most suitable architecture.
- Based on the SRS, multiple design alternatives are created and documented in the **Design Document Specification (DDS)**.

Design

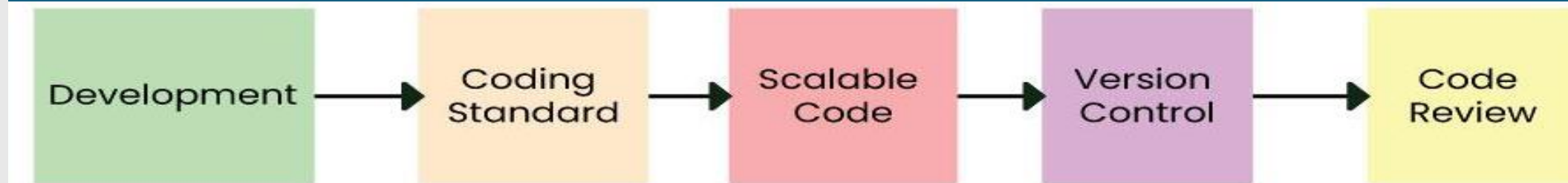
- **Design Evaluation**

- The **DDS** is reviewed and evaluated by stakeholders and market analysts.
- After analyzing all technical and business factors, the **most practical and efficient design** is selected for the implementation phase.

- **Outcome**

- A detailed **system design document** that guides developers during coding.
- Ensures that the software structure is robust, efficient, and meets all user and business requirements.

Development



- The development Phase is when developers start converting the software design into actual, executable code.
- **Objective**
 - The main goal is to develop software that is **functional, efficient, and user-friendly**.
- **Process**
 - Developers use suitable **programming languages** (like Java, Python, C++, etc.) to write the code.
 - Coding is guided by the **Software Design Document (SDD)/ Design Document Specification (DDS)** and follows predefined **coding standards** and guidelines.
 - The SDD/DDS acts as a **roadmap**, ensuring that the code aligns with the system's design and requirements.

Development

- **Code Review**

- Team members perform **regular code reviews** to detect bugs, errors, or inconsistencies early.
- These reviews help maintain **high-quality, reliable, and consistent** code.

- **Internal Testing**

- Basic or **unit testing** is conducted to verify that individual modules or components work correctly.

- **Outcome**

- At the end of this phase, a **working version of the software** is developed.
- It represents the combined effort of planning, analysis, and design from earlier SDLC stages.
- Although not perfect yet, it marks a major step toward delivering the final product.

Integration & Testing



- After the software is developed, **testing** is performed to ensure that it runs smoothly and meets all requirements.
- Though **basic testing** occurs in every phase of SDLC, this stage focuses on **thorough testing** of the complete system.
- The main objectives of this phase are to:
 - Detect and fix all possible **bugs or defects**.
 - **Retest** the system after fixes to ensure proper functioning.
 - Verify that the final product meets the **quality standards** and **requirements** defined in the SRS (System Requirement Specification).

Integration and Testing

Documentation

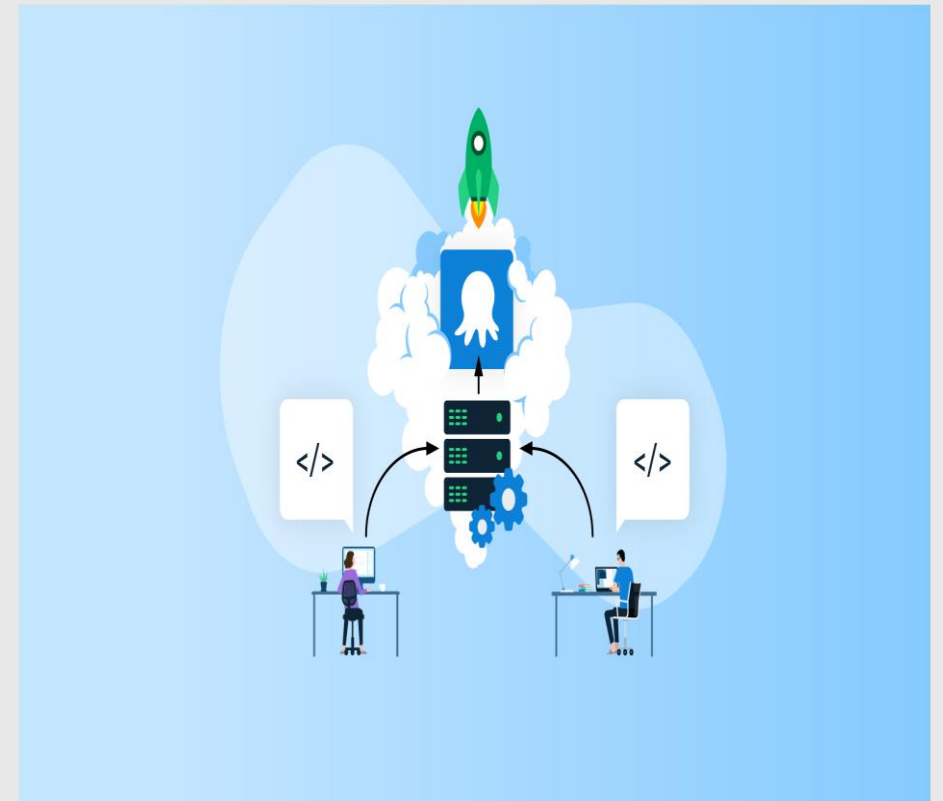
- **Documentation** is a vital part of the software development process.
- It serves as an **information repository** containing details about the system's **functions, design, processes, and maintenance**.
- Good documentation helps:
 - Users understand how to **operate the software**.
 - Developers and maintainers easily **update or troubleshoot** the system in the future.

Training

- **Training** is provided to users or employees to help them understand how to effectively use the new system.
- It aims to **improve performance and efficiency** by enhancing knowledge and skills.
- Training may involve:
 - Demonstrations and hands-on sessions.
 - Manuals or tutorials explaining software usage.
- It helps build user confidence and ensures **smooth adoption** of the software.

Implementation

- After successful testing, the final software product is released in stages according to the organization's plan.
- The product is first tested in a real-world or industrial environment to ensure smooth performance.
- If it performs well, it is released completely for use by end users.
- Feedback is collected from users to identify any improvements or additional features required.
- Based on the feedback, the organization may update or enhance the product.
- Continuous **monitoring and supervision** are done to ensure the software remains stable and secure after release.



Maintenance

- This is the **final and continuous phase** of the Software Development Life Cycle (SDLC).
- It involves **maintaining the system after deployment** to ensure it continues to function smoothly.
- **Bug Fixing:** Errors or issues reported by users are identified and corrected.
- **System Upgrades:** New features and improvements are added to enhance performance and usability.
- **Adaptation:** The software is updated to remain compatible with **new technologies** or **changing business needs**.
- **Performance Monitoring:** The system's performance is regularly reviewed to ensure it meets user expectations.

Maintenance

- After installation, **continuous support and assistance** are provided to resolve issues faced by users.

Support Activities include:

- Providing **helpdesk or technical support** to users.
- Issuing **software patches or updates** to fix bugs or improve functionality.
- **Monitoring system performance** and addressing user feedback promptly.
- The main goal of this phase is to keep the system **reliable, efficient, secure, and up-to-date** throughout its lifecycle.



Approaches to Improve Systems Analysis & Design

- Several approaches have been developed to make the systems analysis and design process more effective and efficient.
- The two most important approaches are:
 - a) **Prototyping**
 - b) **Joint Application Development (JAD)**

a) Prototyping

- Prototyping is a form of **Rapid Application Development (RAD)**.
- It is a **rapid, iterative, and incremental** method where a working model (prototype) of the system is quickly developed.
- Users interact with the prototype and provide feedback.
- The prototype is revised repeatedly until users are satisfied.

Prototyping Process Model

- In this model, the system is **partially developed early**, even before the full analysis is completed.
- This early version helps customers **see and interact with the product** during the development process.
- The process starts by:
 - Interviewing the customers.
 - Preparing an **incomplete, high-level paper model** of the system.
- This document is then used to build the **initial prototype**, which contains only the **basic features** requested by the customer.
- The customer tries the prototype and identifies issues or missing functions.
- Developers **refine and improve** the prototype based on customer feedback.
- This cycle of **feedback -> revision -> improved prototype** continues until:
- The customer is satisfied.
- The prototype meets expectations.
- Once approved, the final working model becomes the basis for the complete system.

Prototyping Process Model

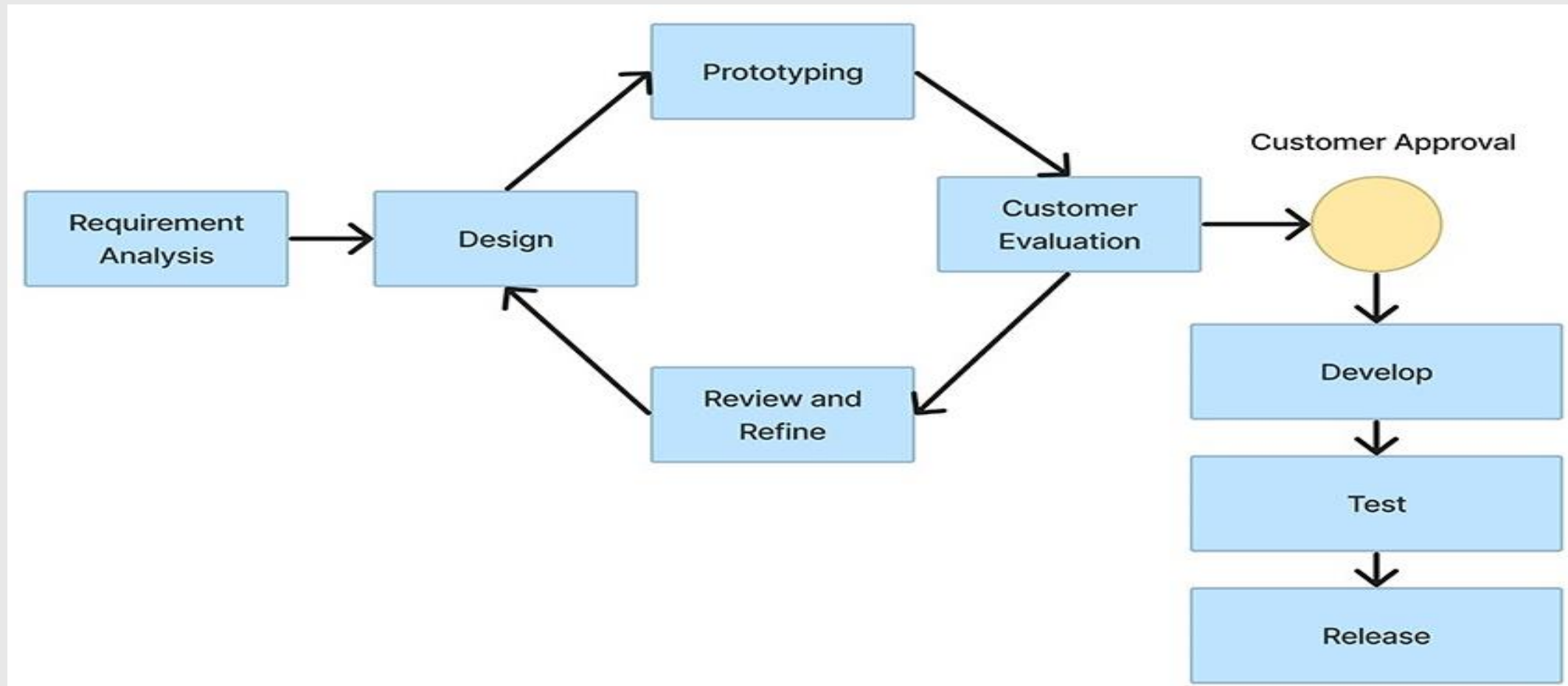


Fig: SDLC phase in prototyping

Advantage of Prototyping model

- Useful when **requirements are uncertain or unclear**.
- Encourages **active user and management participation**.
- Increases **project visibility** and organizational support.
- Users get a working model **quickly**.
- **Errors and missing requirements** are identified early.
- **Testing and training** become easier with the prototype.
- More **natural and user-friendly** development process.
- Best suited for **small to medium-sized projects**.

Disadvantage of prototyping

- **Higher long-term cost** for operation and maintenance.
- May solve the **wrong problem** due to limited initial analysis.
- Final product may have **lower quality** because of speed-focused development.
- Users may mistake the prototype for the final system.

Approaches to Improve Systems Analysis & Design

b) Joint Application Development (JAD)

- JAD is a structured method for **collecting system requirements** and **reviewing system designs**.
- In JAD:
 - Users, managers, and analysts work together.
 - They participate in **intensive, structured meetings** led by a JAD facilitator/session leader.
 - They discuss and agree on system requirements.

Key Points

- Helps gather requirements **quickly and accurately**.
- Ensures **better communication** between users and developers.
- Encourages **teamwork and shared understanding**.
- Saves time and organizational resources.
- Reduces chances of misunderstanding or incomplete requirements.

Joint application development



Fig: JAD phase

Joint application development

Advantages of JAD

- **High-quality software** is produced because users and developers work closely together.
- **Fewer errors** occur since requirements are clarified directly in meetings.
- JAD encourages **better understanding** and **valuable information exchange** between users and developers.
- Faster agreement on requirements because all key members participate at the same time.

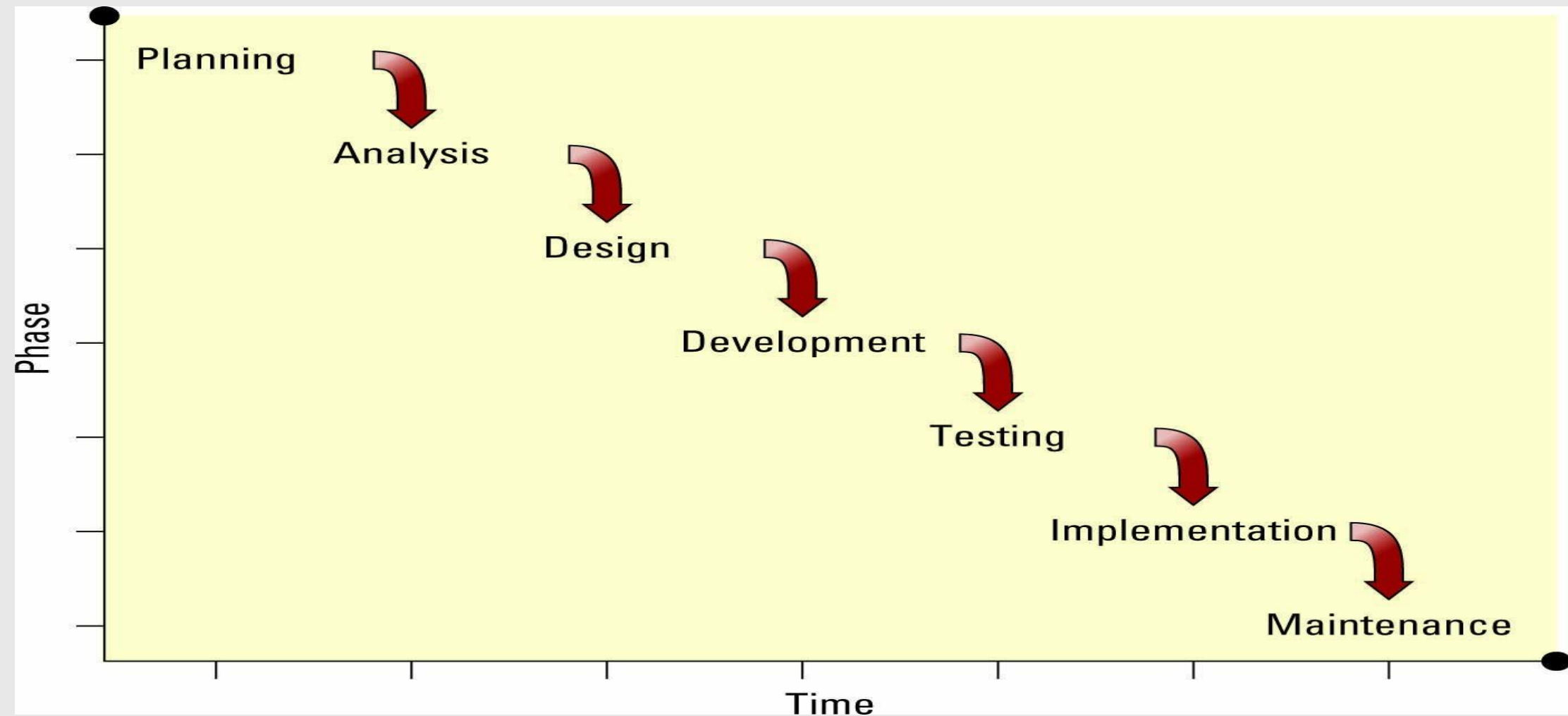
Disadvantages of JAD

- It is a **time-consuming** approach because it requires long and structured workshops.
- It may be difficult to gather all key participants at the same time.
- The process may become expensive if many workshops are needed.

SDLC process model

- Several SDLC models have been developed to improve the way systems are planned, built, and delivered.
- The most widely used SDLC models are:
 - a) Waterfall model
 - b) Incremental process model
 - c) Agile model
 - d) Spiral model
 - e) Rational Unified Model
 - f) Big-Bang Model
 - g) Extreme Programming

Waterfall model



Waterfall model

- Waterfall methodology – A sequence of phases in which the output of each phase becomes the input for the next.
- **Features of the Waterfall Model**
- **Sequential Approach**
 - Each phase is completed fully before the next begins. Progress flows in one direction like a waterfall.
- **Document-Driven**
 - Extensive documentation ensures clarity, consistency, and proper communication throughout all phases.
- **Quality Control**
 - Each phase undergoes verification and validation to ensure the final product meets all requirements.
- **Rigorous Planning**
 - Detailed planning of scope, timelines, and deliverables is done at the beginning and monitored throughout the project.
- **Highly Structured Process**
 - Suitable for environments where a systematic, disciplined development method is required.

Waterfall model

Advantages of the Waterfall Model

- **Simple and Easy to Understand**
 - Clear structure and linear progression.
- **Phase-by-Phase Processing**
 - Each stage is properly defined and follows a fixed order.
- **Clear Milestones**
 - Easy to track progress throughout the lifecycle.
- **Well-Documented**
 - Ensures transparency and easier long-term maintenance.
- **Good Engineering Practices**
 - Encourages “define-before-design” and “design-before-code.”
- **Effective for Small or Well-Known Projects**
 - Works best when requirements are stable and predictable.

Waterfall model

Disadvantages of the Waterfall Model

- **No Feedback Path**
 - Errors discovered late cannot be corrected in earlier stages easily.
- **Difficult to Handle Change Requests**
 - Once the requirements phase is closed, changes become very expensive.
- **No Overlapping of Phases**
 - Inflexible; real-world projects often require parallel development.
- **Limited Flexibility**
 - Rigid structure makes it unsuitable for dynamic or evolving projects.
- **Low Stakeholder Involvement in Later Stages**
 - Clients may only see the final product after development and testing.
- **Late Detection of Defects**
 - Major issues often surface during testing, leading to high fix costs.
- **Long Development Cycle**
 - Sequential execution leads to delays if any phase takes longer than expected.

Waterfall model

When to Use the Waterfall Model?

- **Requirements Are Clear and Stable**
 - No expected changes throughout the project.
- **Minimal Changes Expected**
 - Predictable and fixed development path.
- **Small to Medium Projects**
 - Limited scope and complexity.
- **Predictable and Low-Risk Projects**
 - Risks are known early, and the environment is stable.
- **Projects Requiring High Documentation**
 - Useful in regulated industries—finance, healthcare, government, etc.
- **Client Prefers Linear Development**
 - Some stakeholders want a clear, phase-by-phase roadmap.
- **Limited Resources**
 - Allows controlled resource allocation per phase.

Agile model

- A development method where the software is built in **small parts (iterations / sprints)**, with continuous feedback and improvements.
- Agile is a **flexible** way of developing software.
- Work is done in **small parts (iterations)** instead of building everything at once.
- Agile easily **adapts to changing requirements**, even late in the project.
- It encourages **continuous collaboration** between developers, customers, and stakeholders.
- Teams are **self-organizing** and **cross-functional**, meaning they manage their own work and include all skills needed to complete tasks.

Agile model

- Progress is reviewed frequently, and improvements are made in every iteration.
- It emphasizes:
 - Short development cycles (iterations or sprints)
 - Continuous customer feedback
 - Frequent delivery of usable software
 - Responsiveness to change
 - Collaboration and communication
- **Agile asks: "What does the user want to use next?"**

Agile model



Principle of agile model

- These principles come from the **Agile Manifesto** and guide how Agile teams think, work, and deliver value.
- **Principle 1: Customer Satisfaction through Early and Continuous Delivery**
 - The highest priority is to **satisfy the customer** by delivering valuable, working software **early and continuously**.
- **Principle 2: Welcome Changing Requirements**
 - Changes are welcomed **even late in development**.
 - Agile processes use change to provide the customer with a **competitive advantage**.
- **Principle 3: Deliver Working Software Frequently**
 - Working software is delivered **frequently**—from a **few weeks** to a **few months**, with a strong preference for **shorter cycles**.

Principle of agile model

- **Principle 4: Daily Collaboration**
 - **Businesspeople and developers** must work closely **every day** throughout the project to ensure alignment and clarity.
- **Principle 5: Build Projects around Motivated Individuals**
 - Teams should be built around **motivated individuals**.
 - Give them the right environment, support, and **trust them** to get the job done.
- **Principle 6: Face-to-Face Communication**
 - The most effective way to convey information within a development team is **face-to-face communication**.
- **Principle 7: Working Software as the Primary Measure of Progress**
 - Progress is measured mainly by **working software**, not documentation or plans.

Principle of agile model

- **Principle 8: Sustainable Development**
 - Agile promotes **sustainable development**.
 - Teams (developers, users, and sponsors) should work at a pace that can be **maintained indefinitely**.
- **Principle 9: Focus on Technical Excellence and Good Design**
 - Continuous attention to **technical excellence**, clean code, and good design improves **agility**.
- **Principle 10: Simplicity**
 - Simplicity is essential.
 - It is the art of **maximizing the amount of work not done**—focusing only on what truly matters.

Principle of agile model

- **Principle 11: Self-Organizing Teams**

- The best architectures, designs, and requirements come from **self-organizing teams** that make their own decisions.

- **Principle 12: Regular Reflection and Improvement**

- At regular intervals, the team **reflects** on how to become more effective.
- They then adjust and improve their behavior and processes accordingly.

Agile model

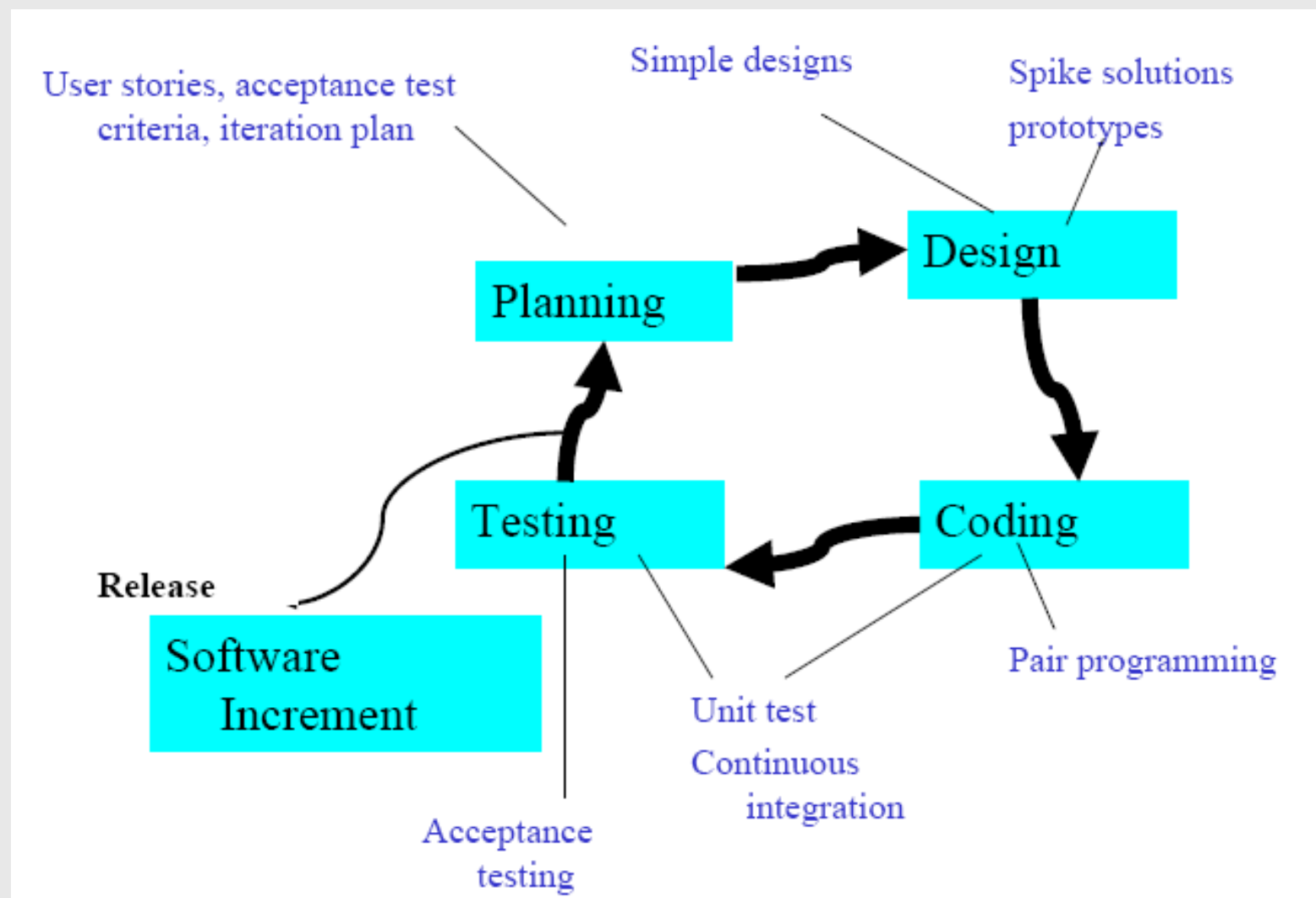
Example (Shopping Application)

- Instead of building the complete shopping app at once, the team develops it step by step.
 - Sprint 1: User registration and login
 - Sprint 2: Product search and product listing
 - Sprint 3: Shopping cart and wish list
 - Sprint 4: Payment and order tracking
- With each sprint, the shopping app becomes completer and more useful for customers.
- **Scenario :**
 - The application is developed step by step with continuous customer interaction and feedback.

Extreme Programming

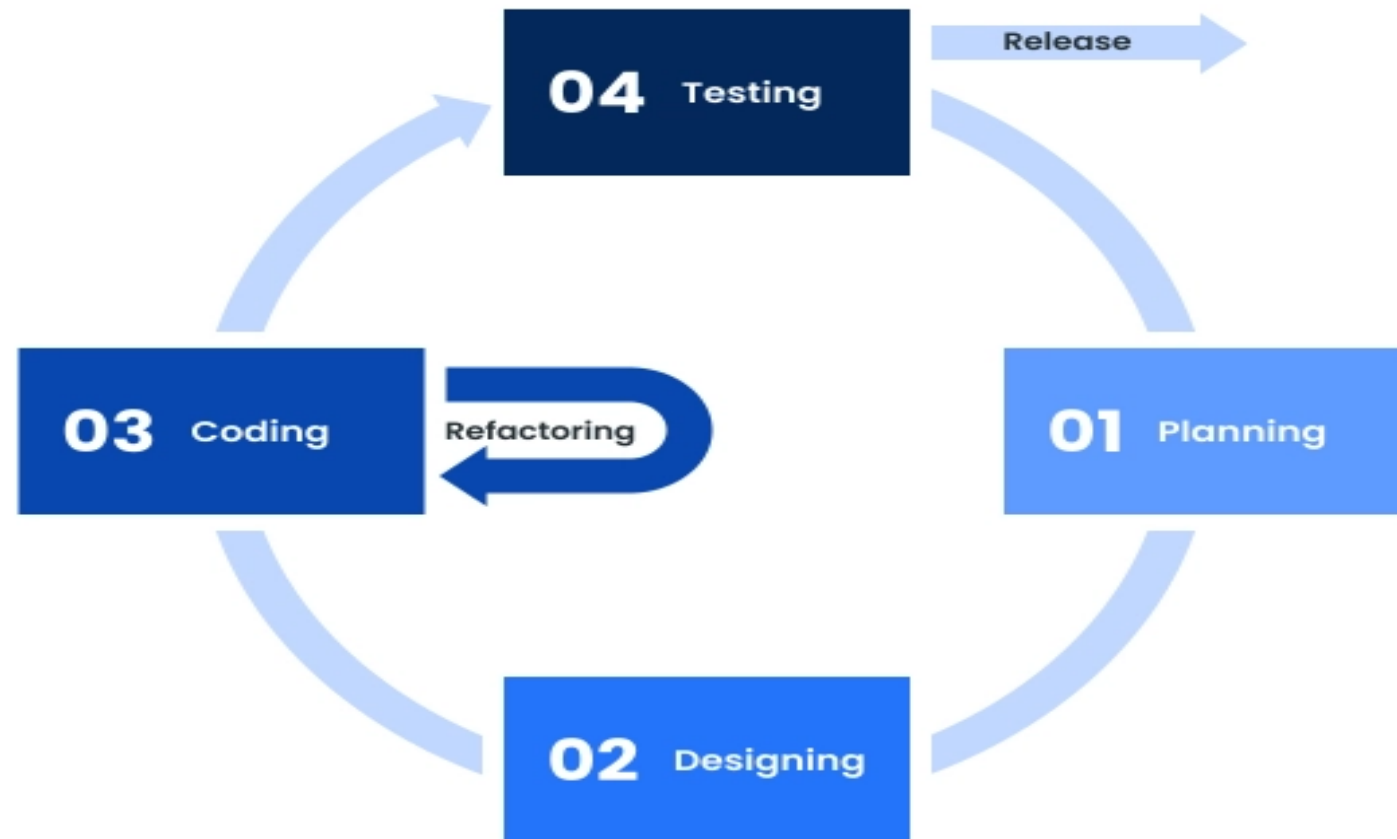
- Extreme Programming (XP) is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation.
- **Key rules of Extreme Programming**
 - Planning
 - Designing
 - Coding
 - Testing

Extreme Programming



Extreme Programming

EXTREME PROGRAMMING (XP)



Extreme Programming

Planning

- Planning begins with user stories, which are short scenarios written by the customer.
- Each story is given a priority value (how important it is).
- The XP team estimates the cost of each story in development weeks.
- If a story takes more than 3 weeks, the customer must split it into smaller stories.
- New stories can be added any time during development.
- Customers and the XP team work together to group stories for the next iteration.
- As development continues, customers may add, remove, or split stories.
- The XP team adjusts the release plan whenever changes happen.

Extreme Programming

Design

- XP prefers a simple design—only what is needed right now.
- Design focuses only on the current user stories.
- Extra or future functionality is not allowed.
- The team identifies object-oriented classes needed for the current system.

Extreme Programming

Coding

- Developers create unit tests for each user story before or during coding.
- After writing the code, each unit must be unit tested.
- **Pair Programming:** Two developers work together at one computer to write production code.
- **Refactoring:** Developers regularly improve and clean the existing code without changing what it does.
- This keeps the code simple, efficient, and easy to maintain.
- Use of coding standards is encouraged so all code looks consistent.

Extreme Programming

Testing

- All unit tests created during coding are run using a testing framework.
- This allows regression testing, which ensures old features still work.
- Integration and validation happen daily, giving constant progress updates.
- Daily testing helps catch problems early.
- Acceptance Tests are created from user stories and are used to confirm whether the story is fully completed.

Spiral Model

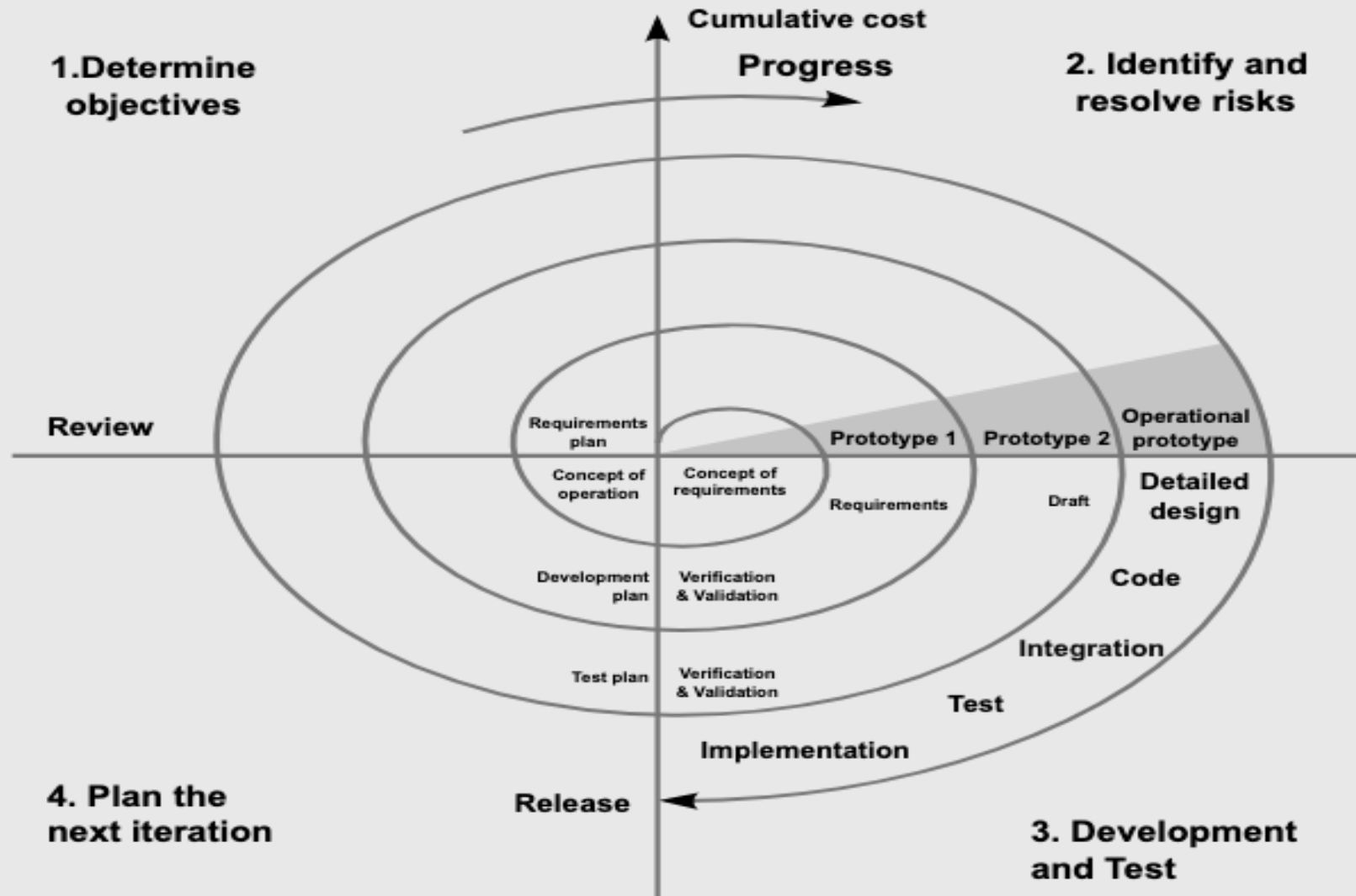
- The **Spiral Model** is a **software development process** in which the project is built in **repeated cycles (spirals)** instead of completing everything at once.
- **You start small -> check for risks -> build a little -> review -> then move to the next bigger cycle.**
- Each new cycle improves the software and adds more features which makes the system becomes **more complete and more refined**.
- It is designed mainly for **large, complex, and high-risk projects**, where requirements may change or are not fully understood at the beginning.
- The main idea behind the model is **risk analysis**- people check for possible problems early to avoid failure later.
- **Spiral asks: "What is the biggest thing that could go wrong next?"**

Spiral Model

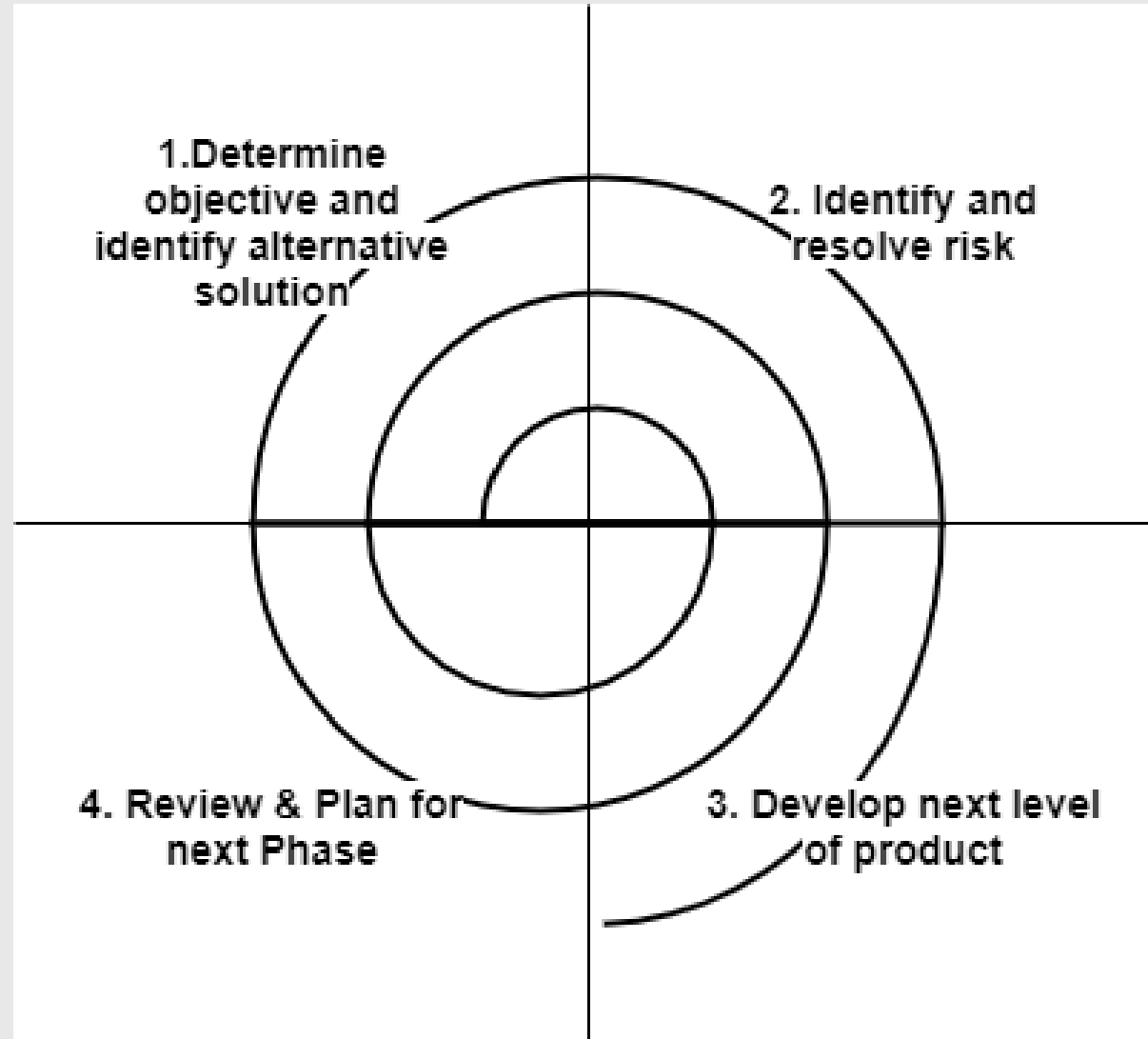
Example (Shopping Application)

- First Spiral: Identify payment security risks and develop basic shopping features
- Second Spiral: Add secure payment gateway and test security
- Third Spiral: Add delivery tracking and improve system performance
- **Scenario**
- Each spiral follows:
 - **Planning -> Risk Analysis -> Development -> Evaluation**
- This model is useful for large and high-risk projects.

Spiral model



Spiral model



Spiral model

1. First Quadrant – Objective Setting

- Identify **goals** for the current iteration.
- Identify **constraints** and **risk factors**.

2. Second Quadrant – Risk Assessment & Reduction

- Analyze each **risk** in detail.
- Take steps to **reduce or remove risks**.
- A **prototype** may be created to check unclear or risky requirements.

3 Third Quadrant – Development & Validation

- Develop the next version of the product.
- Test and validate it to ensure it works properly.

4. Fourth Quadrant – Review & Planning

- Review work with the **customer**.
- Plan what to do in the **next iteration**.
- Each cycle produces a **more complete** version of the software.

Spiral model

Advantages of the Spiral Model

- Excellent **risk management**.
- Suitable for **large** and **critical** projects.
- Allows **changes** at any stage.
- New features can be added easily.
- Early versions of software are available.
- Customers understand the system better over time.
- Cost estimation is easier because work is done in **small parts**.

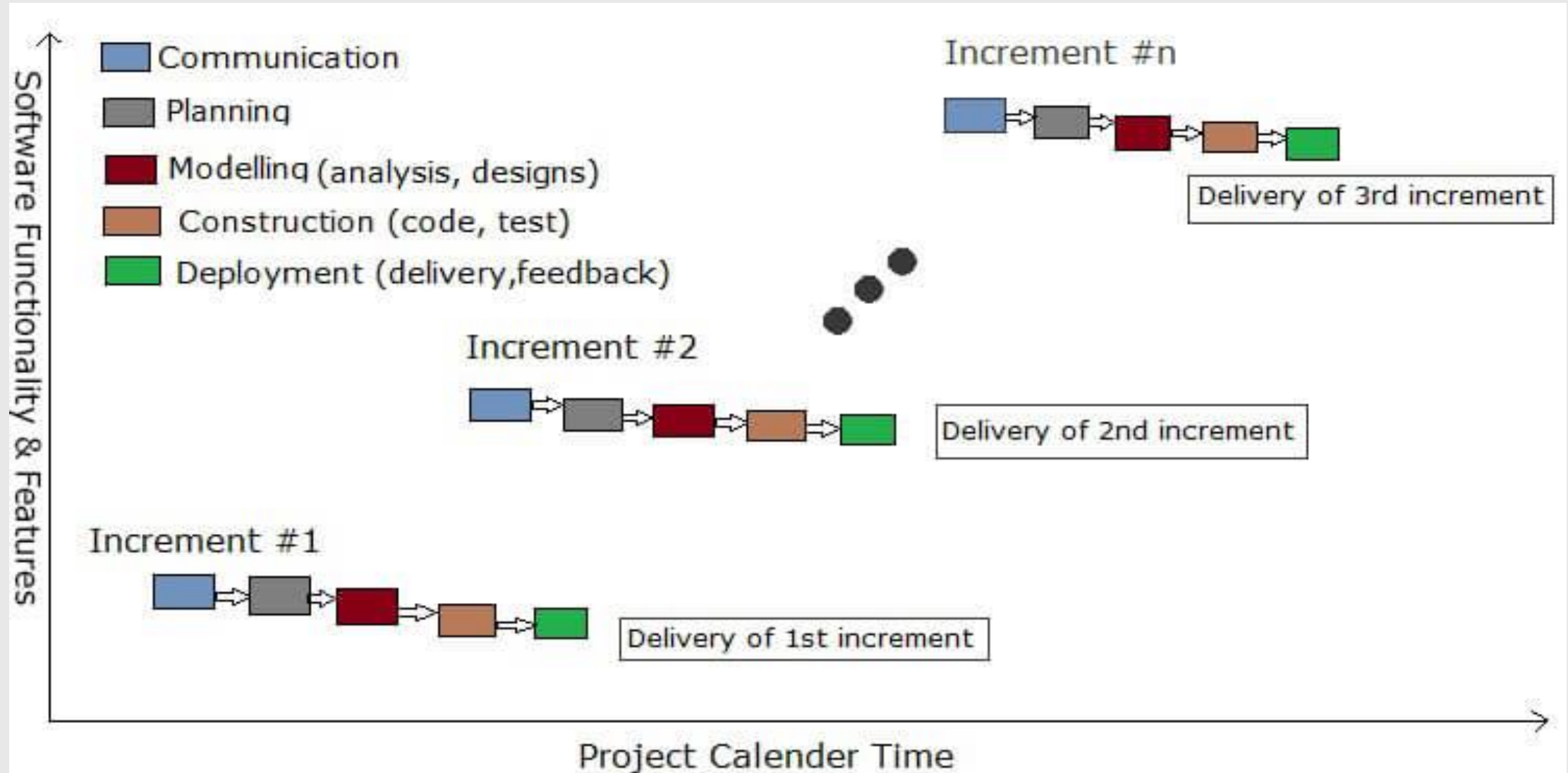
Disadvantages of the Spiral Model

- Difficult to convince customers that the process is under control.
- Needs **experienced** risk evaluators.
- If a risk is missed, problems can occur.
- Works best only for **large and expensive** projects.
- Following the process strictly can be difficult.
- Risk analysis may increase the **overall cost**.
- More documentation and explanation are required.

Incremental Model

- The **Incremental Model** is a software development model in which the complete system is divided into smaller parts called **increments** or **modules**.
- Each increment goes through all phases of the Software Development Life Cycle (SDLC).
- In this model, development happens in multiple cycles, so it is often called a **multi-waterfall model**.
- After each increment, a working version of the software is delivered. New functionality is added step by step until the complete system is developed.
- **Working of Incremental Model**
 - The complete requirements are divided into smaller modules.
 - The first increment develops the core features.
 - After testing and deployment, the next increment is developed.
 - Each new increment is integrated with the previous one.
 - The process continues until the full product is completed.

Incremental Model



Incremental Model

Advantages of Incremental Model

- **Early Working Software**
 - A working product is available early in the development process.
- **Flexible to Changes**
 - Changes in requirements can be implemented more easily.
- **Easy Testing and Debugging**
 - Smaller modules are easier to test and fix.
- **Customer Feedback**
 - Customers can review each increment and provide feedback.
- **Reduced Risk**
 - High-risk modules can be identified and handled early.
- **Lower Initial Cost**
 - Basic functionality can be delivered without waiting for the entire product.
- **Better Resource Management**
 - Development can continue even if all resources are not available initially.

Incremental Model

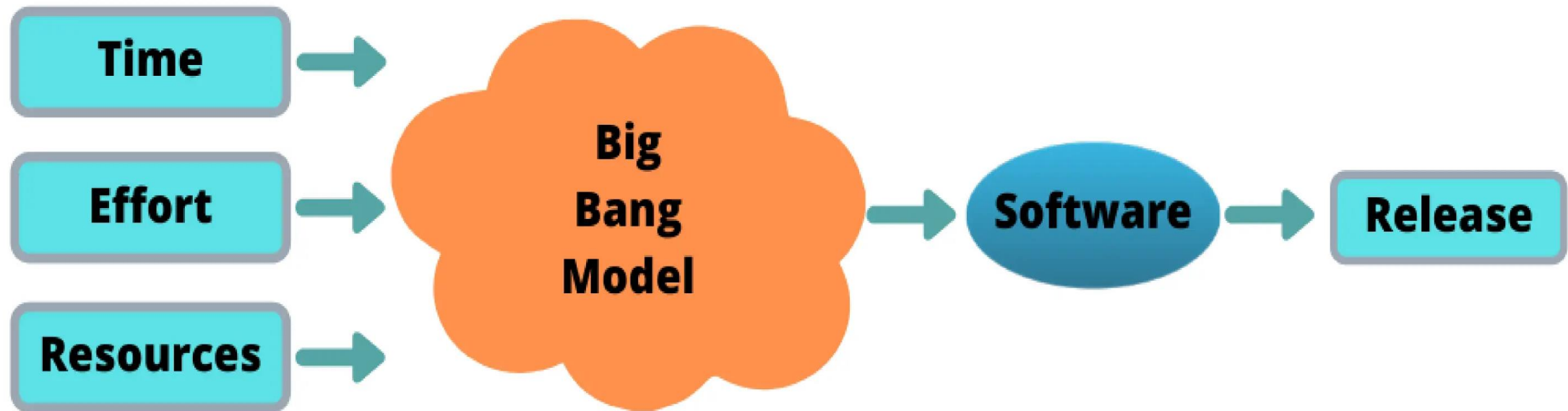
Disadvantages of Incremental Model

- **Requires Good Planning**
 - Proper planning and system design are necessary.
- **Clear Requirements Needed**
 - The complete system requirements should be understood before development starts.
- **Integration Complexity**
 - Combining different modules can sometimes be difficult.
- **Higher Overall Cost**
 - Repeated testing and integration may increase the total cost.
- **Management Complexity**
 - Managing multiple increments may become challenging.

Big Bang Model

- The **Big Bang Model** is a Software Development Life Cycle (SDLC) model that involves little or no planning.
- The Big Bang model requires no or minimal planning. Instead, the development starts with acquiring the inputs, such as:
 - Time.
 - Effort.
 - Resources.
- As soon as the requirements come, they are understood and implemented without any planning.
- This model focuses on software development so developers can jump straight to the implementation to create software as the outcome.
- The primary focus is to code, so this model is more prone to risks than other SDLC models.

Big Bang Model



Big Bang SDLC MODEL

Big Bang Model

When to Use the Big Bang Model?

- The Big Bang Model is not suitable for large or complex software projects because it involves high risk and very little planning.
- However, it can be used for small, temporary, or experimental software projects.
- **This model is suitable when:**
 - The project is small and simple
 - A small team of developers is working together
 - The project is for academic, learning, or practice purposes
 - Requirements are unclear or not fully understood
 - The software is experimental or a dummy project
 - There is no fixed release date for the product

Big Bang Model

Example

- A group of students developing a basic shopping app for a college assignment can use the Big Bang Model because they can directly start coding without detailed planning.
 - In a college assignment scenario, it looks like this:
 - **The Spark:** "Let's build a shopping app!"
 - **The Work:** Everyone opens their laptops and starts coding whatever part they think is cool (one person does the UI, another tries to connect a database).
 - **The Result:** You "bang" the pieces together at the last minute and hope the app runs when the professor looks at it.

Big Bang Model

Advantages of Big Bang Model

- **Simple and Easy to Use**
 - The Big Bang Model is straightforward and easy to understand because it does not follow complex development phases.
- **Minimal Planning Required**
 - Development can start immediately without detailed planning, documentation, or analysis.
- **Flexible Development**
 - Developers have the freedom to make changes during development since there are no strict rules or schedules.
- **Cost-Effective for Small Projects**
 - It requires fewer resources and is suitable for small-scale or temporary projects.
- **Quick Start**
 - Since there is little preparation involved, coding and development can begin quickly.

Big Bang Model

Disadvantages of Big Bang Model

- **Not Suitable for Large Projects**
 - Large projects require proper planning, testing, and management, which this model does not provide.
- **High Risk and Uncertainty**
 - Because of the lack of planning, the project may fail or produce unexpected results.
- **Unclear Requirements Can Increase Cost**
 - If requirements are not properly understood, development may become expensive and time-consuming.
- **Difficult to Manage**
 - Without a structured process, tracking progress and managing the project becomes challenging.
- **Unreliable Model**
 - The final product may not meet customer expectations due to poor planning and unclear goals.

Rational Unified Process (RUP)

- The **Rational Unified Process (RUP)** is a structured software development process used to plan, design, build, test, and deliver software in an organized way.
- It was developed by IBM through Rational Software and is based on the **Unified Modeling Language (UML)**.
- RUP follows an **iterative and incremental approach**, meaning software is developed in small cycles with continuous improvements and testing.
- **Main Features of RUP**
 - Iterative development
 - Incremental delivery
 - Strong focus on testing
 - Risk-driven approach
 - Clear documentation and planning
 - Uses UML diagrams for visual modeling

Rational Unified Process (RUP)

Four Phases of RUP

- **Inception Phase**

- This is the starting phase of the project.
- The development teams determines if the project is worth pursuing and what resources will be needed.
- The main goal of this phase is to understand the business needs and define the basic direction of the project.

- **Main Activities**

- Understanding business requirements
- Defining project goals
- Identifying project scope
- Preparing business case

- **Outcome**

- The project vision becomes clear
- Stakeholders approve the project
- Basic requirements and scope are identified

Rational Unified Process (RUP)

- **Elaboration Phase**

- This phase focuses on detailed planning and system architecture.

- **Main Activities**

- Detailed requirement analysis
 - Risk identification
 - System architecture design
 - Planning resources and schedule

- **Outcome**

- Major risks are reduced
 - Technical foundation is prepared

Rational Unified Process (RUP)

- **Construction Phase**

- In this phase, the actual software is developed.
- The software is designed, written and tested.

- **Main Activities**

- Coding
- Component development
- Continuous testing
- Feature integration

- **Outcome**

- Most system features are completed
- Software becomes stable

Rational Unified Process (RUP)

- **Transition Phase**

- This phase delivers the software to users.
- Final adjustments or updates are made based on feedback from end users.

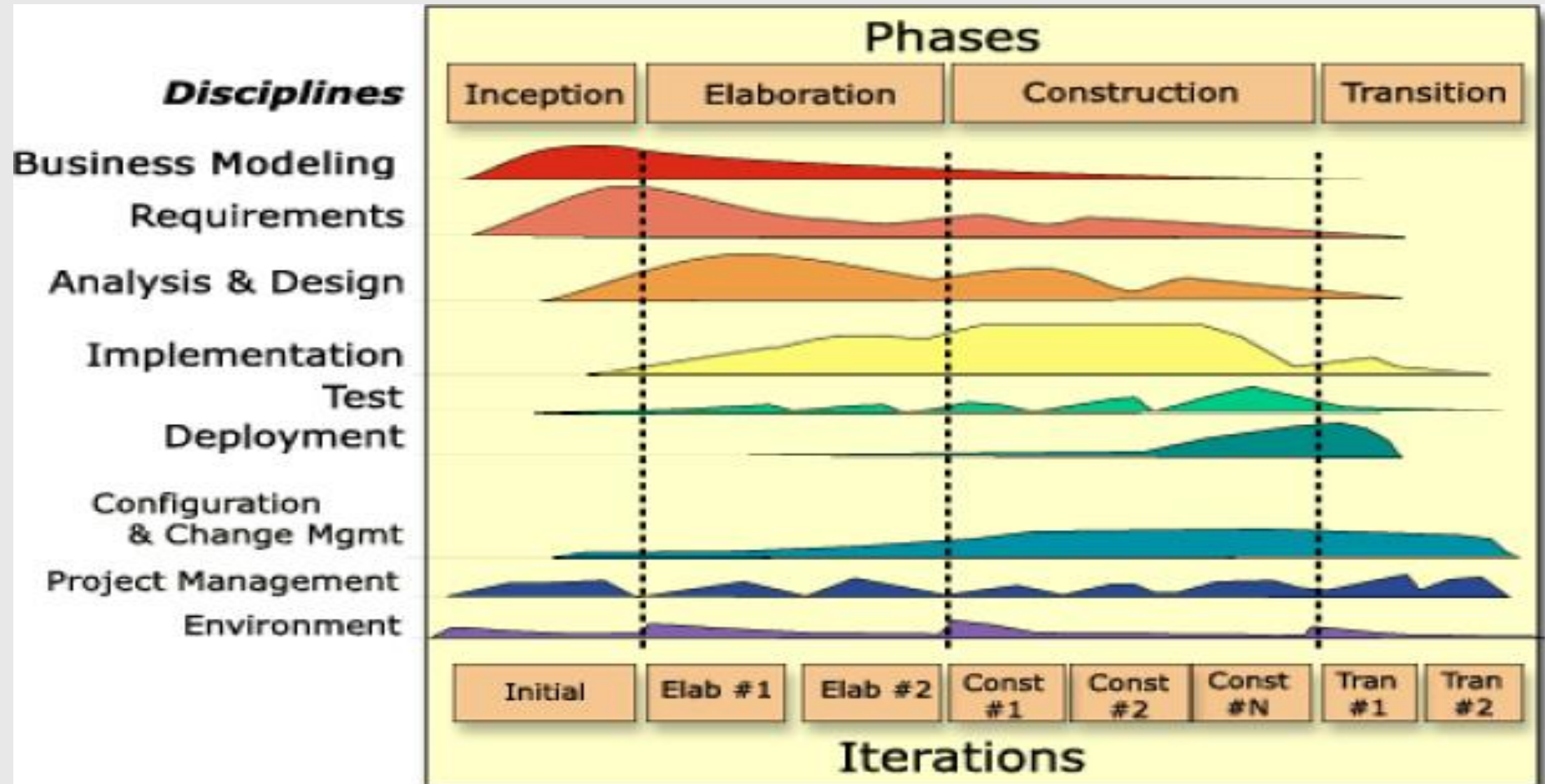
- **Main Activities**

- Deployment
- User training
- Final testing
- Bug fixing

- **Outcome**

- System goes live
- Users start using the software

Rational Unified Process (RUP)



Rational Unified Process (RUP)

Two Types of Iterations in RUP

- **In-Phase Iteration**
- Each RUP phase is completed through **multiple small iterations**.
- Instead of completing a phase in one step, the team continuously improves the work within the same phase.
- In every iteration:
 - Requirements are refined
 - Design is improved
 - Code is enhanced
 - Testing is performed
- Each cycle delivers a **better and more complete version** of the system.

Rational Unified Process (RUP)

Example (Construction Phase)

- Suppose the team is building the **Payment Module** of a shopping application.
 - **Iteration 1:** Create the basic credit card payment form
 - **Iteration 2:** Add expiry date and CVV validation
 - **Iteration 3:** Add payment success notification and error handling
- **Result**
 - The team stays in the same phase but keeps improving the quality and functionality step by step.

Rational Unified Process (RUP)

- **Cross-Phase Iteration**
- Cross-phase iteration means repeating all four RUP phases again for a new version or major improvement of the system.
- **How It Works**
 - A basic version of the software is developed first.
 - Feedback is collected after delivery.
 - The project returns to earlier phases for improvements or new features.
 - This makes RUP flexible and suitable for complex projects.

Rational Unified Process (RUP)

- **Example**
- A company releases **Version 1.0** of a shopping app. Later, customers request a mobile version.
- The team starts another full RUP cycle:
 - **Inception:** Identify mobile app requirements
 - **Elaboration:** Design mobile architecture
 - **Construction:** Develop mobile features
 - **Transition:** Launch the mobile application
- **Result**
 - Each full cycle improves the software's functionality, quality, and user experience.

Rational Unified Process (RUP)

Real-World Example: A Large Banking System

- If a bank wants to replace its core transaction system, they might choose RUP over a "Big Bang" or "Agile" or "Spiral" approach:
 - **Inception:** Decide which banking services (savings, loans, etc.) to include.
 - **Elaboration:** Use **UML Sequence Diagrams** to map out exactly how a transaction moves through the database to ensure no money is ever "lost" due to a logic error.
 - **Construction:** Build the modules for each service one by one.
 - **Transition:** Slowly migrate customer data from the old system to the new one while providing staff training.

Rational Unified Process (RUP)

- **RUP is exactly like building a house.**
 - **Inception (The Idea):** You decide what kind of house to build, check your budget, and see if the land is suitable.
 - **Elaboration (The Blueprint):** You hire an architect to draw **UML diagrams** and pour the foundation. You build the "skeleton" to make sure it won't collapse.
 - **Construction (The Building):** Now that the frame is solid, you put up walls, install plumbing, and finish the rooms one by one.
 - **Transition (The Handover):** You do a final inspection, fix any small scratches, and give the keys to the owner.

Specialized Process Model

- Specialized process models are modified or advanced versions of traditional software development models that are created to solve specific project problems or requirements.
- These models are designed for situations where normal models like Waterfall or Incremental may not be enough.
- They help handle special needs such as:
 - Rapid software development
 - High security and reliability
 - Reusing software components
 - Parallel development activities
 - AI-based automation and testing

Specialized Process Model

- Types of specialized process model are:
 1. **Component Based Development Model**
 2. **Concurrent Development Model**
 3. **Formal Methods Model**
 4. **AI- driven Software Development Model**
 5. **Aspect Oriented Software Development**

Component based development model

- The **Component-Based Development (CBD) Model** develops software by using reusable pre-built components instead of creating everything from scratch.
- **Process of CBD Model:**
 - **Component Identification** - Identify existing components that can be reused.
 - **Component Integration** - Assemble the components to form the system.
 - **Customization & Testing** - Modify and test the integrated components.
 - **Deployment & Maintenance** - Deliver the final system and provide updates.

Component based development model

Advantages

- Faster development by reusing existing components
- Reduces cost and effort
- Encourages modular design and maintainability

Disadvantages

- Integration challenges due to compatibility issues
- Dependency on third-party components
- Limited customization if components are pre-built

Concurrent Development Model

- The **Concurrent Development Model** allows multiple software development activities to occur **simultaneously** rather than in a strict sequence.
- It is ideal for complex systems requiring parallel processing.
- **Phases of Concurrent Model:**
 - Different parts of the system (such as design, coding, and testing) **run in parallel**.
 - Developers work on different modules at the same time.
 - Iterative refinements and feedback loops improve efficiency.

Concurrent Development Model

Advantages

- Shorter development time due to parallel processes
- More flexible than linear models
- Reduces delays in project execution

Disadvantages

- Requires excellent coordination and communication
- More complex to manage than sequential models

Formal method model

- The **Formal Methods Model** is based on **mathematical techniques and logic** to specify, develop, and verify software.
- It is mainly used in **safety-critical systems** (e.g., medical, aviation, and defense software).
- **Steps in Formal Methods Model:**
 - **Mathematical Specification** – Define system requirements using formal logic.
 - **Design & Development** – Implement based on the specifications.
 - **Verification & Proofs** – Use mathematical proofs to ensure correctness.
 - **Testing & Deployment** – Perform additional validation before deployment.

Formal method model

Advantages

- Ensures high reliability and correctness
- Eliminates ambiguities in software specifications
- Reduces critical failures in software systems

Disadvantages

- Complex and difficult to apply
- Requires advanced mathematical expertise
- Costly and time-consuming

Aspect Oriented Software Development Model

- The **Aspect-Oriented Software Development (AOSD) Model** is a software development approach that separates common functionalities such as security, logging, and error handling into independent reusable units called **aspects**.
- These aspects can then be reused across different modules of the software without rewriting the same code repeatedly.

Key Features of AOSD

- **Separates Cross-Cutting Concerns**
 - Common functionalities that affect multiple modules are separated from the main business logic.
 - Example : security, logging, transactions
- **Enhances Modularity and Reusability**
 - Aspects are written once and reused across the system, making the software:
 - Cleaner ,More organized ,Easier to maintain
- **Common in Large-Scale Enterprise Applications**
 - AOSD is widely used in large enterprise systems where many modules require the same common functionalities.
 - Examples: Banking systems , E-commerce platforms ,Hospital management systems

Aspect Oriented Software Development Model

Advantages

- Improves software modularity
- Reduces redundancy in code
- Simplifies maintenance and updates

Disadvantages

- Complex to understand and implement
- Requires specialized tools and frameworks

AI-Driven Software Development Model

- The **AI-Driven Software Development Model** uses Artificial Intelligence (AI) and Machine Learning (ML) in software development to automate tasks such as code generation, bug detection, testing, and maintenance.
- This model helps developers build software faster and more efficiently with the support of AI tools.
- **AI in Software Development**
 - **AI-Powered Code Completion**
 - AI tools can automatically suggest or generate code while developers are programming.
 - **Example**
 - GitHub Copilot suggests code snippets and functions automatically.
 - **Automated Bug Detection and Testing**
 - AI can identify coding errors, security issues, and software bugs automatically.
 - **Example**
 - AI tools can scan an application and detect problems before deployment.
 - **Predictive Maintenance**
 - AI analyzes system behavior and predicts possible failures or maintenance needs in advance.
 - **Example**
 - A banking system can predict server failures before they happen.

AI-Driven Software Development Model

- **Advantages**

- Improves development speed and accuracy
- Reduces human coding errors
- Automates testing and debugging
- Enhances predictive maintenance
- Increases developer productivity

- **Disadvantages**

- Still evolving and not fully reliable
- Requires knowledge of AI and ML
- AI-generated code may contain mistakes
- Can be expensive to implement

The End